

UT 07 – Ejercicios

Ejercicio 17 – Clases y métodos abstractos.

Crea una jerarquía de clases que represente los instrumentos de una orquesta, que se organizan, de forma simplificada, de la siguiente forma:

- Cuerda
 - Frotada
 - Violín (dificultad 9, 4 cuerdas)
 - Violonchelo (dificultad 9, 4 cuerdas)
 - Percutida
 - Piano (dificultad 7, 230 cuerdas)
 - Pulsada
 - Arpa (dificultad 10, 47 cuerdas)
 - Guitarra (dificultad 7, 6 cuerdas)
- Viento
 - Flauta (dificultad 6, madera)
 - Clarinete (dificultad 8, madera)
 - Trompeta (dificultad 7, metal)
 - Tuba (dificultad 8, metal)
- Percusión
 - Xilófono (dificultad 8)
 - Timbales (dificultad 6)
 - Pandereta (dificultad 4)

Ten en cuenta lo siguiente:

- En cada clase debes decidir si debe ser abstracta o no
- En cada clase debes decidir qué métodos son abstractos y qué métodos son implementaciones de métodos abstractos.
- Debes decidir qué constructores se definen en cada clase
- Todas las clases deben heredar de una superclase “Instrumento”, que tiene:
 - Un atributo “propietario” (String), con getter y no setter.
 - Un atributo “dificultad” (int), con getter y no setter.
 - Con un método “void tocar()”. Muestra por consola un mensaje indicando qué instrumento es, y que empieza a sonar.
 - Con un método “void parar()”. Muestra por consola un mensaje indicando qué instrumento es, y que deja de sonar.
 - Un método “bool esMasDifícil(Instrumento instrumento)” que devuelve true si el instrumento es más difícil que el instrumento que se recibe como parámetro.
- Los instrumentos de cuerda:
 - Tienen una propiedad int numeroCuerdas, con getter y no setter.
- Para clasificar los instrumentos de viento, vamos a usar, en lugar de herencia, su material que:
 - Puede tener sólo los valores “MADERA” o “METAL”.
 - Debe implementarse con un tipo enumerado
 - Tendrá getter pero no setter.

Realiza un programa principal que:

- Tenga un método “crearInstrumentos(int numInstrumentos)” que devuelva un array de instrumentos con el tamaño especificado. Este método debe crear los instrumentos de diferentes tipos.
- Tenga un método tocar instrumentos que recorra el array dos veces, una para que los instrumentos empiecen a tocar y otro para que dejen de hacerlo.
- Tenga un método buscarMasDifícil, que recibe el array de instrumentos, y devuelve el más difícil de todos. Para buscar el más difícil, debe usar el método “esMasDifícil”, no comparar los atributos dificultad. Si hay más de un instrumento con la mayor dificultad, se tomará el que aparezca primero en el array.
- Tenga un método mostrarInstrumentos, que muestra los instrumentos del array. Puedes sobrescribir el método toString de los instrumentos para mostrar la información.
- Use los métodos anteriores para crear un array de instrumentos, muestre todos los instrumentos, los ponga a tocar, obtenga el de mayor dificultad y lo muestre, y dejen de tocar.

Ejercicio 18 – Creación de interfaces.

Copia el ejercicio 17 y añade lo necesario para poder afinar los instrumentos que pueden ser afinados. Sólo serán afinables los instrumentos de cuerda (todos), y los timbales. Crea una interfaz “Afinable”, con un método “afinar()”, que se implemente en la clase o clases que consideres más adecuadas. Las implementaciones del método afinar deben mostrar un texto similar a “Afinando <instrumento>”.

Añade al programa principal un método afinarInstrumentos, que recibe el array de instrumentos y afina todos los que se pueden afinar.

Añade, antes de empezar a tocar los instrumentos, la llamada al método afinarInstrumentos.

Ejercicio 19 – Implementación de interfaces – Ordenación de arrays de objetos

Copia el ejercicio 18, y haz los cambios necesarios para:

Antes de llamar al método “mostrarInstrumentos”, ordenar el array por su nivel de dificultad, primero los más difíciles. Este debe ser el “orden natural” de los instrumentos.

Antes de llamar al método “tocarInstrumentos”, ordenar el array por el nombre de su propietario.

Antes de llamar al método “afinarInstrumento” ordene el array por el nombre del instrumento. El nombre del instrumento es el nombre de la clase. Recuerda que el nombre de la clase se puede obtener con el método getSimpleName de la clase Class.